

Java 기본 문법 정리

1. 연산자

1-1. 산술 연산자

숫자를 계산할 때 사용하는 연산자

```
+ // 더하기  
- // 빼기  
* // 곱하기  
/ // 나누기
```

증가 / 감소 연산자

```
x++;
```

위 코드는 다음과 같은 의미이다.

```
x = x + 1;
```

```
x--;
```

위 코드는 다음과 같은 의미이다.

```
x = x - 1;
```

복합 대입 연산자

```
x += 10;
```

위 코드는 다음과 같은 의미이다.

```
x = x + 10;
```

같은 방식으로 사용할 수 있다.

```
x -= 10; // x = x - 10
x *= 10; // x = x * 10
x /= 10; // x = x / 10
```

1-2. 비교 / 논리 연산자

비교 연산자

```
== // 두 값이 같은지 비교
!= // 두 값이 다른지 비교
```

예시

```
if (score == 100) {
    System.out.println("100점");
}
```

논리 AND

```
&&
&
```

두 조건을 모두 만족해야 `true` 가 된다.

```
조건1 && 조건2
```

`&&`

앞의 조건이 `false` 이면 뒤의 조건을 확인하지 않는다.

```
if (age >= 20 && score >= 80) {
}
```

`age >= 20` 이 `false` 라면
`score >= 80` 은 검사하지 않는다.

이를 **Short-Circuit(단락 평가)** 라고 한다.

&

앞의 조건 결과와 관계없이 뒤의 조건까지 모두 확인한다.

조건1 & 조건2

논리 OR

||
|

두 조건 중 하나라도 만족하면 `true`가 된다.

||

앞의 조건이 `true`이면 뒤의 조건을 확인하지 않는다.

조건1 || 조건2

|

앞의 조건 결과와 관계없이 뒤의 조건까지 모두 확인한다.

정리

`&&` : 둘 다 만족해야 함
앞이 `false` → 뒤 조건 확인 X

`&` : 둘 다 만족해야 함
앞의 결과와 관계없이 뒤 조건도 확인

`||` : 하나라도 만족하면 됨
앞이 `true` → 뒤 조건 확인 X

`|` : 하나라도 만족하면 됨
앞의 결과와 관계없이 뒤 조건도 확인

※ 일반적인 조건문에서는 주로 `&&`, `||`를 사용한다.

2. 제어문

프로그램의 실행 흐름을 조건에 따라 제어할 때 사용한다.

대표적으로

```
if
switch-case
```

가 있다.

2-1. if문

조건을 만족하는지 확인하여 실행할 코드를 결정한다.

기본 구조

```
if (조건식) {
    // 조건을 만족할 때 실행
} else {
    // 조건을 만족하지 않을 때 실행
}
```

예시

```
int score = 80;

if (score >= 60) {
    System.out.println("합격");
} else {
    System.out.println("불합격");
}
```

2-2. else if

조건을 여러 개 검사할 수도 있다.

```
if (조건식1) {

} else if (조건식2) {
```

```

} else if (조건식3) {

} else {

}

```

예시

```

int score = 85;

if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else if (score >= 70) {
    System.out.println("C");
} else {
    System.out.println("F");
}

```

조건은 위에서부터 차례대로 검사하며,
하나의 조건을 만족하면 이후 조건은 검사하지 않는다.

2-3. switch-case

하나의 값을 여러 경우와 비교할 때 사용한다.

```

switch (값) {

    case 값1:
        실행문;
        break;

    case 값2:
        실행문;
        break;

    default:
        실행문;

}

```

예시

```
int menu = 1;

switch (menu) {

    case 1:
        System.out.println("커피");
        break;

    case 2:
        System.out.println("우유");
        break;

    default:
        System.out.println("없는 메뉴");
}
```

3. 반복문

같은 코드를 여러 번 반복해서 실행할 때 사용한다.

대표적인 반복문

```
for
while
do-while
```

for문 기본 구조

```
for (초기값; 조건식; 증감식) {
    반복할 코드
}
```

예시

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

출력

```
0  
1  
2  
3  
4
```

while문 기본 구조

```
while (조건식) {  
    반복할 코드  
}
```

조건이 `true` 인 동안 계속 반복한다.

4. Java 이름 작성 관례

Java에서는 클래스명, 변수명, 메소드명을 작성할 때 일반적으로 사용하는 관례가 있다.

클래스명

첫 글자를 **대문자**로 작성한다.

```
Person  
Student  
HelloJava  
MyClass
```

여러 단어를 연결할 경우 각 단어의 첫 글자를 대문자로 작성한다.

```
StudentManager  
HelloJava  
UserAccount
```

이를 **PascalCase**라고 한다.

변수명

첫 글자를 **소문자**로 작성한다.

```
age
score
name
studentName
userAge
```

여러 단어를 연결할 경우 두 번째 단어부터 첫 글자를 대문자로 작성한다.

```
studentName
userAge
totalScore
```

이를 **camelCase**라고 한다.

메소드명

메소드명도 첫 글자를 **소문자**로 작성한다.

```
print()
getName()
calculateScore()
showMessage()
```

메소드명 역시 일반적으로 **camelCase**를 사용한다.

핵심 요약

```
연산자
├─ 산술 : + - * / ++ -- += -=
├─ 비교 : == !=
└─ 논리 : && & || |
```

```
제어문
├─ if
├─ else if
├─ else
└─ switch-case
```

```
반복문
├─ for
└─ while
```


└ do-while

Java 이름 작성 관례

└ 클래스명 → 첫 글자 대문자 (PascalCase)

└ 변수명 → 첫 글자 소문자 (camelCase)

└ 메소드명 → 첫 글자 소문자 (camelCase)